Learning Verified Monitors for Hidden Markov Models

Luko van der Maas and Sebastian Junges

Radboud University, Nijmegen, the Netherlands

Abstract. Runtime monitors observe a running system via sensors and must predict whether the system is at risk of violating a specification, i.e., runtime monitors classify system runs into safe and unsafe behavior. This paper presents a method to synthesise these classifiers, represented as finite automata, if the model is given by a known hidden Markov model. First, we develop the first method to verify the classifiers: We reformulate the verification problem as a search for misclassifications. We then cast the search fo misclassifications into a probabilistic model synthesis problem. Second, we synthesise the monitors inductively by adopting ideas from active automata learning, thereby profiting from the ability to quickly classify individual behavior. The outcome is a flexible synthesis loop that synthesises monitors which provably do not exhibit false negatives and have a controllable false positive rate. Our experiments show the efficacy of a prototype for this loop to synthesise monitors with X states for an HMM with Y states.

TODO abstract does not match intro in terminology.

1 Introduction

Runtime assurance is an essential ingredient in the deployment of safe autonomous systems. In particular, runtime monitors aim to provide assurance by flagging potentially dangerous system behavior, based on the currently available information. More precisely, a monitor receives a stream of observations about the system and outputs a verdict, e.g., an alarm that the system is violating some safety envolope. Various challenges in creating runtime monitors for (semi-)autonomous systems have been identified. Among them is that (1) the state of the system is only partially observable, i.e., the stream of observations comes from sensor readings and does not include all information about the state of a systems, (2) that the behaviour of the system and/or the sensors may be subject to probabilistic uncertainty, (3) that the monitor itself is subject to resource constraints (time, memory, etc), and (4) that monitors are part of the safety-critical functionalities and should therefore be subject to extensive validation [**empty citation**]. The first two challenges can be addressed (partially) by modelling the system as a hidden Markov model (HMM), challenges (3) and (4) can be partially addressed by representing a monitor as, e.g., a (small) finite

TODO add HMM / image of system feeding a trace into a monitor?

Fig. 1.

state machine. Concretely, this paper focusses on the following question: *Is a given finite state machine a correct monitor for a given HMM?* The paper studies the complexity of this problem, provides a practical verification procedure, and embeds this verification procedure into a learning framework.

What are HMMs? In this paper, we assume that the system including its sensors is adequately described as a discrete Hidden Markov Model (HMM). HMMs, like Markov chains (MCs), describe system behavior subject to probabilistic uncertainty. Paths are sequences of states that describe system executions. HMMs extend MCs by labelling their states with colours (aka *observations*) Intuitively, the observations can be used to model the information that the monitor receives. In HMMs, every path can be lifted to a sequence of observations, which we call a *trace*. The trace associated to a system execution describes the information received by the monitor.

What is the monitoring problem on HMMs? The standard verification task for MCs is to compute the probability that a path violates some temporal specification. The monitoring question on HMMs is to compute the *conditional* probability of a path violating a specification, given that the path prefix matches some given observation trace. This conditional probability is the *risk of a trace*. A monitor thus maps traces to risks. The risks can be computed in polynomial time, either via a reduction to model checking, or by a (forward) filtering that tracks a distribution over the current system state.

What are correct monitors? Many monitors are used to raise an alarm if the risk of a trace exceeds some threshold. We call a monitor *correct* (for an HMM, a specification, and a threshold) if it correctly classifies the risk of every trace as below or above the threshold. Thus, the traces are partitioned into safe and unsafe traces. A correct monitor raises an alarm exactly on the unsafe traces. To check whether a monitor is correct, we will utilise two checks: First, we check whether the monitor correctly raises an alarm on every unsafe trace (no false negatives) and second, we check whether the monitor correctly raises no alarm on every safe trace (no false positives). In terms of a formal languages, these are two inclusion checks.

Illustrative example. TODO I guess here is a good place ot make things more concrete.

Complexity. To study the decision problem for correct monitors, we consider a simpler problem that asks whether there is a bounded trace in the HMM that induces a risk above a given threshold, i.e., whether there is a false negative. [1] The problem is clearly in NP. We show that this simplified problem is already strongly NP-hard (Thm 2) and the corresponding optimization problem, that asks to

[1] The results also apply to checking for false positives.

approximate the risk of the trace with the heighest risk, is APX-hard (Thm 2). Intuitively, this means that there is little hope for an efficient approximation algorithm.

Reduction to probabilistic synthesis. While the problem is thus in general not tractable, we suggest to utilise recent advances in the synthesis of probabilistic systems. We concentrate on proving the absence of false negatives, a similar reduction works for showing the absence of false positives. First, for a single trace, computing the risk can be reduced to computing a reachability probability in a MC that is intuitively a kind of product between a DFA that accepts exactly the trace and the HMM [**empty citation**]. Thus, it is natural to reduce our problem to the question: Is there a DFA (accepting exactly one trace, accepted by our monitor) [2] such that the probability in the product MC exceeds a threshold. If the answer is no, then the monitor has no false negatives. This decision problem can be solved using (exact) probabilistic model synthesis methods [**empty citation**] as implemented in the tool Paynt [**empty citation**].

Learning monitors with active automata learning. Ultimately, we do not only want to verify a monitor, but we want to construct it. Above, we already mentioned that the monitors are formal languages. When considering bounded trace, these monitors are regular, and can thus be captured using DFAs. Inspired by [**empty citation**], we investigate learning these monitors using active automata learning [**empty citation**]. To use active automata learning, we must provide a *membership oracle* that decides for a single trace whether the risk of the trace exceeds a threshold and an *equivalence oracle* that decides whether a given hypothesis monitor is indeed correct. The membership oracle can be implemented by computing the risk for a single trace as above and comparing it to a threshold, while the equivalence oracle can be implemented using the two inclusion checks as above.

Putting it together. In Fig. 2, we outline our proposed framework to learn a monitor. TODO explain

Practical considerations in using active automata learning. Practically, some modifcations are important:

- Finite horizon
- dont care
- combining conformance and equivalence queries?

Empirical evaluation. TODO potentially summarise results of the empirical eval here.

Contributions. In summary, this paper provides a first framework to learn runtime monitors — encoded as finite automata — that are verified to be correct

[2] Alternatively, in Section **??**, we show that we can construct a product of an acyclic DFA accepting sets of traces and the HMM. However, this yields partially observable MDP, where we must search for memoryless policies, similar to a construction in [**empty citation**]. This problem is equivalent to the synthesis problem and can therefore also be solved using Paynt [**empty citation**].

TODO add our framework

Fig. 2.

with respect to a given HMM. The main contributions are as follows: (1) We study the monitor verification problem and show the hardness of the problem, even in an approximative setting. (2) We reduce the monitor verification problem to a probabilistic synthesis problem that search for a counterexample. In the absence of a counterexample, we can conclude that the monitor is correct. (3) We learn monitors by using the verifier above to answer equivalence queries in conjunction with off-the-shelf active automata learning. (4) We demonstrate the feasibility and limitations of both the verifier and the learner on a series of benchmarks. Our prototype can handle HMMs with thousands of states, up to a hundred observations, and monitors with over 100 states.

2 Preliminaries

2.1 Hidden Markov Models

Definition 1. *A (discrete-time) Markov chain (MC) is a tuple* $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ *where:*

- S *is a countable, nonempty set of states,*
- $\mathbf{P} : S \times S \to [0, 1]$ *is the transition probability function such that for all states* s *:*

$$\sum_{s' \in S} \mathbf{P}(s, s') = 1$$

- ι *is the initial state, and*
- AP *is a set of atomic propositions, and*
- $L : S \to 2^{AP}$ *is a labeling function.*

Additionally, observations can be added to an MC by adding a Set of observations O*, together with an observation function* $obs\colon S \to O$*.*

From [2]

[3].

Definition 2. *The probability measure* $Pr^{\mathcal{M}}$ *of an MC* $\mathcal{M}$ *is the unique probability measure following from the canonical* σ*-algebra associate with mc.*

Definition 3. *Given an MC* $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$*, the paths conforming to a trace* τ *are defined as*

$$\mathrm{Paths}(\tau) = \{\pi \in \Pi^{fin} \mid obs(\pi) = \tau\}$$

[3] HMMs can also be defined with a distribution over colours: This leads to more concise HMMs, however, there is a polynomial-time transformation to the simpler HMMs we use in this paper.

Definition 4. *Given an MC $\mathcal{M}$, the cylinder set* $\mathrm{Cyl}(\pi)$ *of a finite path π is the set of all infinite extensions of π in $\mathcal{M}$.*

2.2 Monitors

Definition 5. *A deterministic finite automaton (DFA) $\mathcal{A}$ is a tuple $\mathcal{A} = (Q, \Sigma, \delta, \iota, F)$ where:*

- *Q is a finite set of states,*
- *Σ is an alphabet,*
- *$\delta \colon Q \times \Sigma \rightharpoonup Q$ is a (partial) transition function,*
- *ι is the initial state, and*
- *$F \subseteq Q$ is a set of accept (or: final) states.*

From [2]

Definition 6. *The language $\mathcal{L}(()\mathcal{A})$ of a DFA $\mathcal{A}$ is the set of all words accepted by the $\mathcal{A}$.*

2.3 Problem Statement

We have two problems we want to solve. The no false alarms problem and the no missed alarms problem. They have slightly different problem statements and therefore also slightly different solutions.

2.4 No false alarms

All traces τ from the MC which are in the monitor have at least a probability $p \in I$ of reaching the good state.

Definition 7. *Given a Markov chain $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ with observations in the set of colors C, a specification DFA $\mathcal{A} = \left(Q_{\mathcal{A}}, 2^{AP}, \delta_{\mathcal{A}}, \iota_{\mathcal{A}}, F_{\mathcal{A}}\right)$, AP, and a DAG monitor DFA $\mathcal{B} = (Q_{\mathcal{B}}, C, \delta_{\mathcal{B}}, \iota_{\mathcal{B}}, F_{\mathcal{B}})$. For all traces τ from the MC and in the monitor, the probability of $\pi \in \mathcal{L}(\mathcal{A})$ given that π conforms to τ is in a given interval I*

$$\operatorname*{in}^{I}_{\tau \in \mathcal{L}(\mathcal{M}) \cap \mathcal{L}(\mathcal{B})} Pr^{\mathcal{M}}(\pi \in \mathcal{L}(\mathcal{A}) \mid \pi \in \mathrm{Cyl}(\mathrm{Paths}(\tau)))$$

2.5 No missed alarms

All traces τ which reach the good state with probability $p > t$, are in the monitor. By taking the contrapositive, we can instead show that all traces τ which are not in the monitor reach the good state with probability $p \leq t$.

Definition 8. *Given a Markov chain $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ with observations in the set of colors C, a specification DFA $\mathcal{A} = \big(Q_{\mathcal{A}}, 2^{AP}, \delta_{\mathcal{A}}, \iota_{\mathcal{A}}, F_{\mathcal{A}}\big)$, AP, and a DAG monitor DFA $\mathcal{B} = (Q_{\mathcal{B}}, C, \delta_{\mathcal{B}}, \iota_{\mathcal{B}}, F_{\mathcal{B}})$. For all traces τ from the MC and not in the monitor, the probability of $\pi \in \mathcal{L}(\mathcal{A})$ given that π conforms to τ is in a given interval I*

$$\operatorname*{in}^{I}_{\tau \in \mathcal{L}(\mathcal{M}) \cap \overline{\mathcal{L}(\mathcal{B})}} Pr^{\mathcal{M}}(\pi \in \mathcal{L}(\mathcal{A}) \mid \pi \in \mathrm{Cyl}(\mathrm{Paths}(\tau)))$$

Monitor verification problem

3 Monitor Verification

3.1 Complexity

Theorem 1. *The monitor verification problem is ???-complete.*

Membership?

Hardness For hardness, we study the following simpler problem. CTP = Conditional trace probability.

Definition 9 (CTP Decision Problem). *Given an HMM $\mathcal{M}$ with colors C, a unary encoded horizon H, and a threshold $\lambda \in (0, 1]$, is there a trace $\tau \in C^H$ such that $Pr^{\mathcal{M}}(\Diamond G \mid \tau) \geq \lambda$.*

Lemma 1. *The CTP Decision Problem is NP-hard.*

SJ: What about the probability of a trace being zero?

The result holds even for $\lambda = 1$ and acyclic HMMs. We illustrate the reduction from 3CNF. Consider a 3CNF formula $\varphi = \bigwedge c_1 \dots c_m$ over n variables X, with clause $c_j = \ell_j^1 \vee \ell_j^2 \vee \ell_j^3$ and each literal $\ell_j^i \in \{x, \neg x \mid x \in X\}$. We transform this into a CTP instance with $\lambda = 1$ and an HMM with colours $C = \{\#, \bot, \top\}$. We will create an HMM such that there is a trace with CPT $= 1$ iff there is a satisfying assignment to the 3CNF formula.

Before we give a formal definition of the HMM, we give the following intuition, supported by Fig. 3. We represent assignments $\alpha \colon X \to \{0, 1\}$ by traces through the HMM $\#\# \cdot \alpha(x_0) \cdot \# \cdot \alpha(x_1) \cdot \dots \cdot \#$. This trace matches with m paths through the HMM, one for each clause. That is, the HMM randomly selects a clause with probability $\frac{1}{m}$ in the initial state and then evaluates this clause c_j with respect to an assignment. A path through a clause reads variable x_i in state $s_{i,j}$ coloured with $\#$ and transitions to $s_{i,j}^{\top}$ and $s_{i,j}^{\bot}$, both with probability 0.5. State $s_{i,j}^{\top}$ has colour $\top$, while $s_{i,j}^{\bot}$ has color $\bot$. Thus, the conditional probability to reach $s_{i,j}^{\top}$ is $\frac{1}{m}$ if $\alpha(x_i) = \top$ and 0 otherwise. Then, if the literal x_i (or $\neg x_i$) occurs in clause c_j, we transition from $s_{i,j}^{\top}$ (or $s_{i,j}^{\bot}$, resp.) to $s'_{i+1,j}$, otherwise,

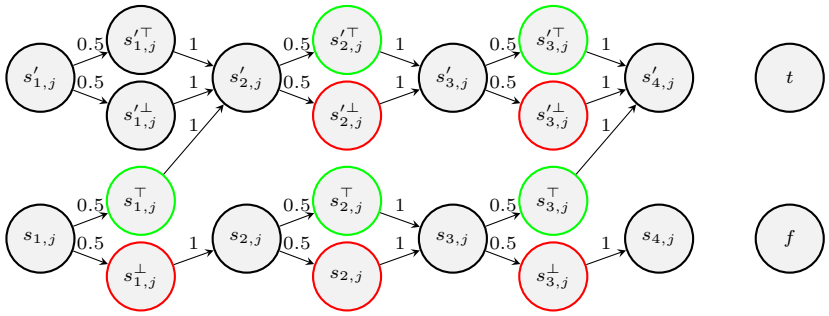

Fig. 3. The main gadget for the reduction of Lemma 1, demonstrated for the clause $x_1 \vee \neg x_3$ and variables x_1, x_2, x_3.

we transition to $s_{i+1,j}$. The idea is that (conditionally) reaching state $s'_{i,j}$ (with positive probability) means that the partial assignment $\alpha(x_1), \ldots, \alpha(x_i)$ satisfies clause c_j, while (conditionally) reaching $s_{i,j}$ (with positive probility) means that the clause is either unresolved or unsatisfied given the partial assignment.

We construct the following HMM $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ and set $\lambda = 1$:

- $S = \{s_{i,j}, s^{\bot}_{i,j}, s^{\top}_{i,j}, s'_{i,j}, s'^{\bot}_{i,j}, s'^{\top}_{i,j} \mid i \in \{1, \ldots, n+1\}, j \in \{1, \ldots, m\}\} \cup \{s_\iota\}$.
- $\mathbf{P}$ is given such that for all $i \in \{1, \ldots, n\}, j \in \{1, \ldots, m\}$:
  - $\mathbf{P}(s_\iota, s_{1,j}) = \frac{1}{m}$,
  - $\mathbf{P}(s_{i,j}, s^{\top}_{i,j}) = \mathbf{P}(s_{i,j}, s^{\bot}_{i,j}) = \frac{1}{2} = \mathbf{P}(s'_{i,j}, s'^{\top}_{i,j}) = \mathbf{P}(s'_{i,j}, s'^{\bot}_{i,j})$,
  - $\mathbf{P}(s^{\top}_{i,j}, s'_{i+1,j}) = [x_i = \ell^k_j$ for some k$]$, $\mathbf{P}(s^{\top}_{i,j}, s_{i+1,j}) = [x_i \neq \ell^k_j$ for all k$]$,
  - $\mathbf{P}(s^{\bot}_{i,j}, s'_{i+1,j}) = [\neg x_i = \ell^k_j$ for some k$]$, $\mathbf{P}(s^{\bot}_{i,j}, s_{i+1,j}) = [\neg x_i \neq \ell^k_j$ for all k$]$
  - $\mathbf{P}(s'_{n+1,j}, t) = 1 = \mathbf{P}(s_{n+1,j}, f)$
- $c(s^{\top}_{i,j}) = (s'^{\top}_{i,j}) = \top$, $c(s^{\bot}_{i,j}) = (s'^{\bot}_{i,j}) = \bot$, $c(s_{i,j}) = c(s'_{i,j}) = \# = c(s_\iota) = c(t) = c(f)$.
-

Definition 10 (binary-CTP Decision Problem). *Given an HMM $\mathcal{M}$ with colors C, a binary encoded horizon H, and a threshold $\lambda \in (0, 1]$, is there a trace $\tau \in C^H$ such that $Pr^{\mathcal{M}}(\Diamond G \mid \tau) \geq \lambda$.*

I have no clue how to show that is PSPACE hard... The PSPACE hardness would somehow have to use the long horizons.... I also dont believe it is important.

Approximability Approximating the verification of monitors is also hard, in the sense that it is hard to approximate the maximal risk that a monitor admits.

The maximal risk problem is an optimization problem

Definition 11 (CTP Optimization Problem).

Lemma 2. *The CTP Optimization Problem is APX-hard.*

We show this by means of an L-reduction from MAX-3SAT, which is an inapproximable and APX-hard problem [**empty citation**]. The construction follows along the reduction for the NP-hardness.

3.2 Reduction to Policy Synthesis

Definition 12. *A Markov decision process is a tuple* $\mathcal{M} = (S, Act, \mathbf{P}, \iota, AP, L)$ *where*

- S *is a countable set of states,*
- Act *is a set of actions,*
- $\mathbf{P} : S \times Act \times S \to [0,1]$ *is the transition probability function such that for all states* $s \in S$ *and actions* $\alpha \in Act$*:*

$$\sum_{s' \in S} \mathbf{P}(s, \alpha, s') \in \{0,1\}$$

- ι *is the initial state,*
- AP *is a set of atomic propositions, and*
- $L : S \to 2^{AP}$ *is a labeling function.*

From [2]

4 Problem Statements

We have two problems we want to solve. The no false alarms problem and the no missed alarms problem. They have slightly different problem statements and therefore also slightly different solutions.

4.1 No false alarms

All traces τ from the MC which are in the monitor have at least a probability $p \in I$ of reaching the good state.

Definition 13. *Given a Markov chain* $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ *with observations in the set of colors* C*, a specification DFA* $\mathcal{A} = \left(Q_{\mathcal{A}}, 2^{AP}, \delta_{\mathcal{A}}, \iota_{\mathcal{A}}, F_{\mathcal{A}}\right)$*,* AP*, and a DAG monitor DFA* $\mathcal{B} = (Q_{\mathcal{B}}, C, \delta_{\mathcal{B}}, \iota_{\mathcal{B}}, F_{\mathcal{B}})$*. For all traces* τ *from the MC and in the monitor, the probability of* $\pi \in \mathcal{L}(\mathcal{A})$ *given that* π *conforms to* τ *is in a given interval* I

$$\operatorname*{in}^{I}_{\tau \in \mathcal{L}(\mathcal{M}) \cap \mathcal{L}(\mathcal{B})} Pr^{\mathcal{M}}(\pi \in \mathcal{L}(\mathcal{A}) \mid \pi \in \mathrm{Cyl}(\mathrm{Paths}(\tau)))$$

4.2 No missed alarms

All traces τ which reach the good state with probability $p > t$, are in the monitor. By taking the contrapositive, we can instead show that all traces τ which are not in the monitor reach the good state with probability $p \leq t$.

Definition 14. *Given a Markov chain* $\mathcal{M} = (S, \mathbf{P}, \iota, AP, L)$ *with observations in the set of colors* C*, a specification DFA* $\mathcal{A} = \left(Q_{\mathcal{A}}, 2^{AP}, \delta_{\mathcal{A}}, \iota_{\mathcal{A}}, F_{\mathcal{A}}\right)$*,* AP*, and a DAG monitor DFA* $\mathcal{B} = (Q_{\mathcal{B}}, C, \delta_{\mathcal{B}}, \iota_{\mathcal{B}}, F_{\mathcal{B}})$*. For all traces* τ *from the MC and not in the monitor, the probability of* $\pi \in \mathcal{L}(\mathcal{A})$ *given that* π *conforms to* τ *is in a given interval* I

$$\operatorname*{in}^{I}_{\tau \in \mathcal{L}(\mathcal{M}) \cap \overline{\mathcal{L}(\mathcal{B})}} Pr^{\mathcal{M}}(\pi \in \mathcal{L}(\mathcal{A}) \mid \pi \in \mathrm{Cyl}(\mathrm{Paths}(\tau)))$$

5 Outline

See fig. 4 for an outline of the algorithm.

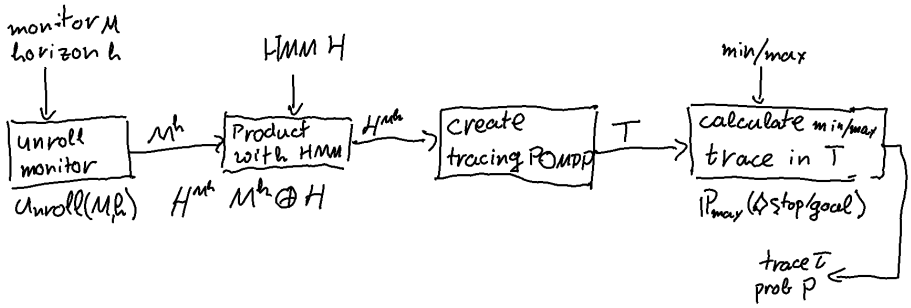

Fig. 4. Outline of the algorithm.

6 Algorithm

Our algorithm will proceed in four steps. First, we take the MC product [2] of the MC $\mathcal{M}$ and the specification $\mathcal{A}$. Then, we will combine the product MC with the monitor into an MDP. Next we will add end transitions to this MDP for all states that are accepted by the underlying monitor. Lastly, we use Paynt [1] to model check the created MDP.

6.1 Applying the specification to the MC

We take the MC product as defined by Baier and Katoen [2] to create the product MC $\mathcal{M} \oplus \mathcal{A}$.

Intuition The created product MC is the original MC, but every good state is labeled.

6.2 Constructing the product MDP

To create the MDP, we first create its states. The states are the product of the states of the product MC and the monitor. These are denoted as $\langle s, q_{\mathcal{B}} \rangle$ for $s \in S_{\mathcal{M} \oplus \mathcal{A}}$, and $q_{\mathcal{B}} \in Q_{\mathcal{B}}$. As actions in the product MDP we take the observations C of the MC $\mathcal{M}$.

The transitions in the MDP are defined here in two steps. First we create a not-yet-normalized transition function P' on the states in the MDP. Next, we

normalize the transition function by creating redirecting all remaining probability mass of an action in a state towards the initial state in P''.

$$P'(\langle s,q\rangle, c, \langle s',q'\rangle) = \begin{cases} P(s,s') & \text{if } obs(s') = c \wedge q' = \delta(q,c) \\ 0 & \text{otherwise} \end{cases}$$

$$P''(\langle s,q\rangle, c, \langle s',q'\rangle) = \begin{cases} 1 - \sum_{s''\in S}\sum_{q''\in Q} P'(\langle s,q\rangle, c, \langle s'',q''\rangle) & \text{if } s' = \iota_{\mathcal{M}\oplus\mathcal{A}} \wedge q' = \iota_{\mathcal{B}} \\ P'(\langle s,q\rangle, c, \langle s',q'\rangle) & \text{otherwise} \end{cases}$$

Lastly, the atomic propositions are $\{\circledcirc_{\mathcal{A}}, \circledcirc_{\mathcal{B}}\}$, where accepting states in the product MC are labeled by $\circledcirc_{\mathcal{A}}$ and accepting states in the monitor are labeled by $\circledcirc_{\mathcal{B}}$.

Intuition The created MDP can be seen as the original product MC $\mathcal{M}\oplus\mathcal{A}$, where we can choose which observation to observe, but only if the monitor contains the transition labeled by the observation.

6.3 Adding end transitions

We add two new states to the MDP, the *goal* and *stop* states, each only having a self loop. Next, we will add an *end* action to every state with the label $\circledcirc_{\mathcal{B}}$. If the state also contains the $\circledcirc_{\mathcal{A}}$ label, we direct the *end* action to the *goal* state, otherwise we direct the *end* action to the stop state.

Intuition Since we are talking about finite traces in the monitor, we need to give the MDP a way to end a path. This is done by taking the *end* action. If we take the end action in a good state of the product MDP, we reach the *goal* state, if it was a bad state, then we reach the *stop* state.

6.4 Model checking using Paynt

We create a POMDP which will be model checked by Paynt [1] for reachability of the goal state. The observations in this POMDP are the states of the monitor. Using this POMDP we calculate

$$\mathbb{P}_{\max}[\diamond goal]$$

to find the maximum probability and

$$1 - \mathbb{P}_{\max}[\diamond stop]$$

to find the minimum probability. Finally, we can verify this upper and lower bound with the interval I.

Intuition In order to find the traces in the monitor which give the maximum and minimum probability of reaching a good state, we associate a trace τ with a scheduler σ_τ in an MDP. Such a scheduler is called consistent.

Definition 15. *A* consistent scheduler *is a scheduler which generates only one trace*

$$\exists_{\tau \in \Sigma_{obs}} \ \forall_{\pi \in Paths(\sigma)} \ obs(\pi) = \cdots \iota \tau e^{\star}$$

where ι is the observation of the initial state and e is the observation of the goal and stop state.

By model checking the created POMDP, we don't yet ensure that every scheduler is consistent, instead we only ensure semi-consistence.

Definition 16. *A semi-consistent scheduler σ is a scheduler which has the following property*

$$\exists_{\tau \in \Sigma_{obs}} \ \forall_{\pi \in Paths(\sigma)} \ obs(\pi) = (\iota \tau^{+})^{\star}$$

where τ^{+} can be a different prefix of τ for every loop in $()^{\star}$.

Thus, a semi-consistent scheduler does not have to take and end action. However, by maximizing the probability of reaching either the *goal* or *stop* state, we ensure that if the *end* action can be taken, it will be taken. And if a path takes the *end* action, almost surely any other path will also take the *end* action at some point. This property ensures that when maximizing semi-consistent schedulers we will always get a consistent scheduler, and thus a single trace.

7 Learning Monitors

8 Experiments

9 Related

LTLf

10 Conclusion

References

[1] Roman Andriushchenko et al. "PAYNT: A Tool for Inductive Synthesis of Probabilistic Programs". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 856–869. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8_40.

[2] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

[3] Christian Hensel et al. "The Probabilistic Model Checker Storm". In: *International Journal on Software Tools for Technology Transfer* 24.4 (Aug. 1, 2022), pp. 589–610. ISSN: 1433-2787. DOI: 10.1007/s10009-021-00633-z. URL: https://doi.org/10.1007/s10009-021-00633-z (visited on 09/02/2024).

[4] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. "The Glory of the Past". In: *Logics of Programs*. Ed. by Rohit Parikh. Berlin, Heidelberg: Springer, 1985, pp. 196–218. ISBN: 978-3-540-39527-0. DOI: 10.1007/3-540-15648-8_16.

A Alternative formulations of the problem

Instead of giving a specification DFA, we could instead give a TL specification [4] which is evaluated for every state. Since TL has both future and past operators both now reachability, $\diamond\!\!\!\!\cdot\, l$, and soon reachability, $\bigcirc^n \diamond\!\!\!\!\cdot\, l$, for a label l can be expressed. While using a specification DFA can only express now reachability.

The only impact this reformulation of the problem would have is in step 1. Here, instead of taking the product MC, we check if the above described TL formula has a probability above a certain threshold. If that is the case we mark it with the $\circledcirc_{\mathcal{A}}$ label. This MC is then used for the subsequent steps.

B Reformulation with monitors

C Experiments

We build our tool in the python programming language on top of the model checker Storm [3] and the POMDP model checker PAYNT [1]. This tool includes both the ToVer verifier algorithm and the learning algorithm employing ToVer for equality queries.

TODO: Write further description of the tool.

Benchmarks We evaluate both the ToVer verifier and the learning algorithm. We evaluate the ToVer verifier using the following questions:

$\mathbf{Q}^1_{VM}$ How does the runtime of ToVer verifier depend on its parameters?
$\mathbf{Q}^2_{VM}$ How much does the ToVer verification depend on PAYNT?
$\mathbf{Q}^3_{VM}$ Which PAYNT strategy is the most efficient in the ToVer benchmarks?

Maybe not a full question?

We also evaluate the learning algorithm by looking at the following questions:

$\mathbf{Q}^1_{L}$ Does ToVer learning guarantee a better bound then learning using a sampling EQ?
$\mathbf{Q}^2_{L}$ How do the ToVer learning monitors and sampling learning monitors compare in size?
$\mathbf{Q}^3_{L}$ Do monitors learned without a horizon limit in MQ generalize to outside the horizon?

Again, maybe less relevant?

C.1 Benchmarks

References

[1] Roman Andriushchenko et al. "PAYNT: A Tool for Inductive Synthesis of Probabilistic Programs". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Cham: Springer International Publishing, 2021, pp. 856–869. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8_40.

[2] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

[3] Christian Hensel et al. "The Probabilistic Model Checker Storm". In: *International Journal on Software Tools for Technology Transfer* 24.4 (Aug. 1, 2022), pp. 589–610. ISSN: 1433-2787. DOI: 10.1007/s10009-021-00633-z. URL: https://doi.org/10.1007/s10009-021-00633-z (visited on 09/02/2024).

[4] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. "The Glory of the Past". In: *Logics of Programs*. Ed. by Rohit Parikh. Berlin, Heidelberg: Springer, 1985, pp. 196–218. ISBN: 978-3-540-39527-0. DOI: 10.1007/3-540-15648-8_16.